DUNDALK INSTITUTE OF TECHNOLOGY
INSTITIÚID TEICNEOLAÍOCHTA DHÚN DEALGAN

Continuous Assessment Cover Sheet

Student Name: Shauna McGuinness	**Student Number: D00243983**

Programme: BSc (Hons) in Computing Systems and Operations (Software Development and DevOps)	**Stage:3**	**Complete Student Checklist:** Re-read brief ☐ References and Bibliography ☐ Proofread ☐
Module: Data Structures & Algorithms		
Due Date: 11/01/2026	**No. Pages:**	
Lecturer(s) Name: Michelle Graham		
Assignment No. and/or Description/Topic: Reflection and Design Justification		**Mode of Submission:** Softcopy ☐ Hardcopy ☐

DECLARATION: I declare that:

- This work is entirely my own, and no part of it has been copied from any other person's words or ideas, except as specifically acknowledged through the use of inverted commas and in-text references;
- No part of this assignment has been written for me by any other person except where such collaboration has been authorised by the lecturer(s) concerned;
- I have not used generative artificial intelligence (AI) (e.g. ChatGPT) unless it has been permitted by the lecturer(s) concerned;
- I understand that I am bound by DkIT Academic Integrity Policy. I understand that I may be penalised if I have violated the policy in any way;
- This assignment has not been submitted for any other module at DkIT or any other institution, unless authorised by the relevant Lecturer(s);
- I have read and abided by all of the requirements set down for this assignment.

SIGNATURE SHAUNA MCGUINNESS **DATE 10/01/2026**

Lecturer's Comments:

Provisional Mark : _____ **Lecturers Signature :** _____ **Date:** _____

1. Introduction

This project involved developing an analytics system for a streaming platform that processes user interaction logs to generate useful insights about platform usage. While the overall system addresses data filtering, deduplication, and user activity analysis, my individual contribution focused on implementing time-based data filtering, analysing user viewing spans, and integrating these features into the main application.

One of the main challenges I encountered was deciding how to handle time-based queries efficiently without making the solution overly complex. Supporting timestamp range queries and calculating the longest continuous viewing span required balancing performance with simplicity, especially given the size constraints of the dataset.

This report outlines the dataset used and the assumptions made, followed by a discussion of the algorithms I implemented and the reasons behind these choices. It then examines the data structures used, analyses their complexity, and reflects on the design decisions and trade-offs made throughout the project.

---

2. Data Description

The dataset consists of interaction logs from a streaming platform. Each record contains seven fields: Interaction ID, Session ID, User ID, Timestamp, Content ID, Action Type (PLAY, PAUSE, SKIP, STOP, LIKE), and Duration (seconds). The intended dataset size is fewer than 5,000 records, which influenced the design approach by allowing a focus on clarity and correctness while still considering performance.

Two datasets were used during testing, each containing approximately 50 interaction records. These datasets were used to support integration testing of the application.

Several assumptions were made during implementation: timestamps are unique across all interactions, allowing them to be safely used as keys, and sessions represent continuous viewing periods. As a result, a user's viewing span can be calculated using their first and last interaction timestamps without explicitly modelling breaks**.**

---

3. Implemented Algorithms

This section describes only the algorithms I personally implemented as part of the group project.

3.1 Timestamp Range Query

Operation:

This operation retrieves all interaction records that fall within a specified timestamp range. It supports analytics tasks such as analysing platform activity during specific time windows or identifying peak usage periods.

Algorithm Used:

A binary search tree–based range query implemented using a custom CustomTreeMap. Interactions are indexed by timestamp, and a subMap(from, to) operation performs an in-order traversal to collect only those records whose timestamps fall within the given range.

Reason for Choice:

I chose a tree-based structure because it allows timestamp range queries to be performed efficiently once the data has been indexed. During early development, I initially considered scanning the full dataset for each query, but this quickly became

inefficient when multiple time-based queries were required. Using a binary search tree ensures that only relevant portions of the data are traversed.

Alternative Considered and Rejected:

A linear scan over all interactions was considered but rejected because it always requires O(n) time, regardless of how small the requested time window is.

3.2 Longest Viewing Span Calculation

Operation:

This algorithm determines which user has the longest continuous viewing span, calculated as the difference between their first and last recorded interaction timestamps.

Algorithm Used:

A single-pass grouping approach using a hash-based map structure. As each interaction is processed, the earliest and latest timestamps for each user are updated. Once all interactions have been processed, the user with the largest time difference is identified.

Reason for Choice:

Only two timestamp values per user are required, allowing the problem to be solved efficiently in O(n) time without sorting. This approach is both time-efficient and memory-conscious.

Alternative Considered and Rejected:

Sorting interactions for each user or performing pairwise timestamp comparisons was rejected due to unnecessary computational overhead.

3.3 Application Integration Logic

Operation:

The main application is responsible for loading the datasets, executing the analytics operations, and outputting the results.

Algorithm Used:

A structured control-flow sequence that invokes the timestamp range query and longest viewing span calculation after the data has been loaded.

Reason for Choice:

Separating the analytics logic from the main application flow improves readability and makes the system easier to test and extend.

---

4. Data Structures Used

A CustomTreeMap, implemented as a node-based binary search tree, was used to store and index interaction records by timestamp. Each node stores a key–value pair along with references to left and right child nodes, allowing the tree to maintain its keys in sorted order based on natural comparison. This structure was implemented in a generic form, enabling timestamp values to be used as keys without restricting the tree to a fixed data type. The primary reason for choosing this structure was to support efficient timestamp range queries, as in-order traversal of the tree allows records within a specific time window to be retrieved without scanning the entire dataset.

A hash-based map structure was used when calculating the longest viewing span for each user. This structure maps user IDs to their earliest and latest interaction timestamps. Hashing allows expected constant-time access when updating these

values during processing. The structure was conceptually generic, with user IDs used as keys and timestamp pairs stored as values. This approach was chosen because it enables user activity spans to be calculated efficiently in a single pass through the dataset. If a built-in Java HashMap is used, collision handling is managed internally by the Java Collections Framework.

A dynamic array (ArrayList-style structure) was used to collect and return query results, such as interactions returned from timestamp range queries. Dynamic arrays automatically resize as elements are added, providing fast append operations and efficient indexed access. This structure was chosen because the number of results is not known in advance, and it integrates cleanly with both custom data structures and built-in Java collections when producing output.

---

5. Complexity Analysis

The timestamp range query implemented using the CustomTreeMap executes in O(log n + k) average time, where *n* is the total number of interaction records and *k* is the number of results returned within the specified range. This efficiency comes from the tree's ordered structure, which allows irrelevant branches to be skipped during traversal. In the worst case, if the binary search tree becomes unbalanced, the time complexity may degrade to O(n + k). The space complexity of this operation is O(k), as only the matching results are stored.

The longest viewing span calculation operates in O(n) time using a single-pass grouping approach, where each interaction is processed once. The space complexity is O(u), where *u* is the number of unique users, since only two timestamps are stored per user. An alternative approach involving sorting interactions per user would have increased the time complexity to O(n log n) and was therefore rejected.

The application integration logic introduces minimal additional overhead, as it primarily coordinates data loading and invokes the analytics operations, resulting in O(n) time and O(1) extra space.

---

6. Design Decisions and Trade-offs

One of the main design decisions I made was to use an unbalanced binary search tree rather than implementing a self-balancing structure such as an AVL or Red-Black tree. While a balanced tree would provide better worst-case performance guarantees, implementing and debugging it would have added significant complexity to the project. Given that the dataset size was limited to fewer than 5,000 records, I felt that the simpler approach was more appropriate and easier to reason about, while still providing acceptable performance in practice.

When calculating viewing spans, I chose a hash-based structure so that user interactions could be grouped efficiently in a single pass through the dataset. This approach uses additional memory to store per-user timestamps, but it significantly improves runtime performance compared to nested comparisons or repeated scans. I considered this trade-off acceptable given the relatively small dataset size.

Overall, the design prioritised clarity and correctness over maximum optimisation. This helped ensure the system remained efficient enough for the intended use case while keeping the code readable, testable, and easier to maintain.

7. Testing and Validation

I implemented three test classes to verify the correctness of my contributions: CustomTreeMapTest validates the BST operations including put, get, remove, and the critical subMap range query method. All eight test cases passed successfully, confirming that the tree maintains correct ordering and retrieves ranges accurately. TimestampRangeTest verifies that the timestamp range query returns the correct interactions for a given time window. The test confirmed that querying the 08:30-09:00 window correctly returned 15 interactions from Dataset_1.

LongestSpanTest confirms that the longest viewing span calculation correctly identifies the user with the maximum time between first and last interaction. The test validated that U002 was correctly identified with a 90-minute span

These tests demonstrate that the CustomTreeMap structure functions correctly and that time-based queries produce accurate results.

8. Appendix

A. BST Range Query Snippet

```
public CustomTreeMap<K, V> subMap(K fromKey, K toKey) {
  CustomTreeMap<K, V> result = new CustomTreeMap<>();
  collectInRange(root, fromKey, toKey, result);
  return result;
}
```

B. Operation vs Complexity Summary

Operation	Algorithm	Time Complexity	Space Complexity
Timestamp range query	BST subMap	O(log n + k) avg	O(k)
Longest viewing span	Single-pass grouping	O(n)	O(u)
Application integration	Control-flow sequencing	O(n)	O(1)

C. Test Results Screenshots

```
=== CustomTreeMap Test ===

Test 1: Basic put and get
Size: 5
Get 3: three
Get 7: seven
? Test 1 passed

Test 2: Keys in sorted order
Keys: [1, 3, 5, 7, 9]
? Test 2 passed

Test 3: First and last key
First key: 1
Last key: 9
? Test 3 passed

Test 4: SubMap (range query)
SubMap [3-7]: [3, 5, 7]
SubMap size: 3
? Test 4 passed

Test 5: Remove
Removed value: five
Size after remove: 4
Keys after remove: [1, 3, 7, 9]
? Test 5 passed

Test 6: Replace existing key
Old value: three
New value: THREE
? Test 6 passed

Test 7: String keys (simulating timestamps)
Time keys: [2025-11-23 08:01, 2025-11-23 09:15, 2025-11-23 10:30, 2025-11-23 11:45]
Time range [09:00-11:00]: [2025-11-23 09:15, 2025-11-23 10:30]
? Test 7 passed

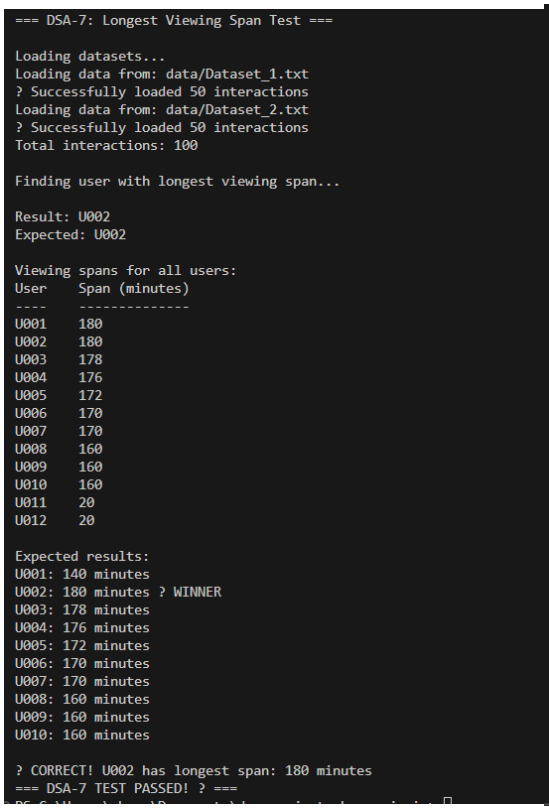
=== ALL TESTS PASSED! ? ===
CustomTreeMap is working correctly!
```

```
Loading data from: data/Dataset_1.txt
? Successfully loaded 50 interactions
Loading data from: data/Dataset_2.txt
? Successfully loaded 50 interactions
Checking for duplicates...\n
DUPLICATE FOUND:
  Dataset 1: I016       S003    U003    2025-11-23 08:31        PLAY    C003    150
  Dataset 2: I053       S003    U003    2025-11-23 08:31        PLAY    C003    150

DUPLICATE FOUND:
  Dataset 1: I025       S005    U005    2025-11-23 08:49        PLAY    C005    180
  Dataset 2: I055       S005    U005    2025-11-23 08:49        PLAY    C005    180

DUPLICATE FOUND:
  Dataset 1: I043       S001    U001    2025-11-23 09:25        PLAY    C001    210
  Dataset 2: I081       S001    U001    2025-11-23 09:25        PLAY    C001    210

Total duplicates found: 3
Expected: 3 (I016=I053, I025=I055, I043=I081)
\nUser interaction counts after deduplication:
U001: 11
U002: 12
U003: 10
U004: 10
U005: 9
PS C:\Users\chaun\Documents\dsa_project_chauna_jamie>
```



```
=== DSA-7: Longest Viewing Span Test ===

Loading datasets...
Loading data from: data/Dataset_1.txt
? Successfully loaded 50 interactions
Loading data from: data/Dataset_2.txt
? Successfully loaded 50 interactions
Total interactions: 100

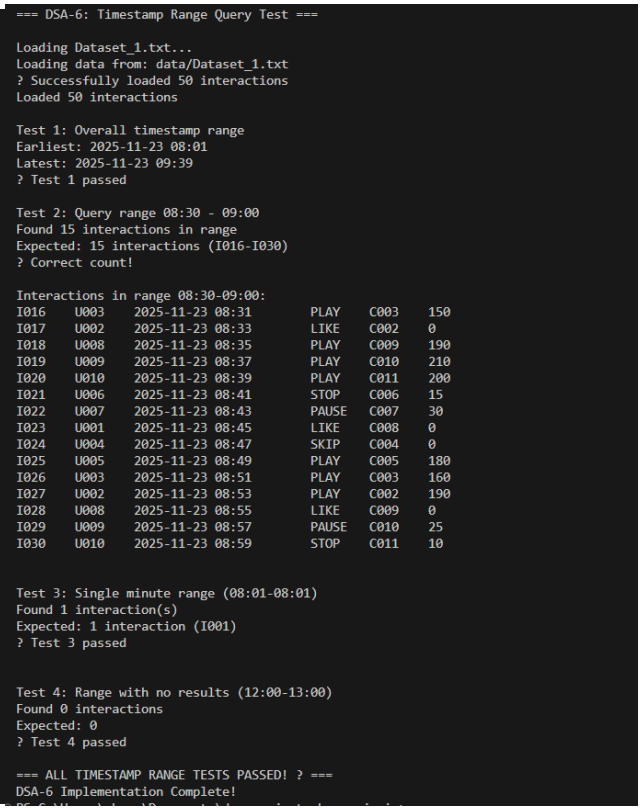
Finding user with longest viewing span...

Result: U002
Expected: U002

Viewing spans for all users:
User    Span (minutes)
----    --------------
U001    180
U002    180
U003    178
U004    176
U005    172
U006    170
U007    170
U008    160
U009    160
U010    160
U011    20
U012    20

Expected results:
U001: 140 minutes
U002: 180 minutes ? WINNER
U003: 178 minutes
U004: 176 minutes
U005: 172 minutes
U006: 170 minutes
U007: 170 minutes
U008: 160 minutes
U009: 160 minutes
U010: 160 minutes

? CORRECT! U002 has longest span: 180 minutes
=== DSA-7 TEST PASSED! ? ===
```



```
=== DSA-6: Timestamp Range Query Test ===

Loading Dataset_1.txt...
Loading data from: data/Dataset_1.txt
? Successfully loaded 50 interactions
Loaded 50 interactions

Test 1: Overall timestamp range
Earliest: 2025-11-23 08:01
Latest: 2025-11-23 09:39
? Test 1 passed

Test 2: Query range 08:30 - 09:00
Found 15 interactions in range
Expected: 15 interactions (I016-I030)
? Correct count!

Interactions in range 08:30-09:00:
I016    U003    2025-11-23 08:31        PLAY    C003    150
I017    U002    2025-11-23 08:33        LIKE    C002    0
I018    U008    2025-11-23 08:35        PLAY    C009    190
I019    U009    2025-11-23 08:37        PLAY    C010    210
I020    U010    2025-11-23 08:39        PLAY    C011    200
I021    U006    2025-11-23 08:41        STOP    C006    15
I022    U007    2025-11-23 08:43        PAUSE   C007    30
I023    U001    2025-11-23 08:45        LIKE    C008    0
I024    U004    2025-11-23 08:47        SKIP    C004    0
I025    U005    2025-11-23 08:49        PLAY    C005    180
I026    U003    2025-11-23 08:51        PLAY    C003    160
I027    U002    2025-11-23 08:53        PLAY    C002    190
I028    U008    2025-11-23 08:55        LIKE    C009    0
I029    U009    2025-11-23 08:57        PAUSE   C010    25
I030    U010    2025-11-23 08:59        STOP    C011    10


Test 3: Single minute range (08:01-08:01)
Found 1 interaction(s)
Expected: 1 interaction (I001)
? Test 3 passed


Test 4: Range with no results (12:00-13:00)
Found 0 interactions
Expected: 0
? Test 4 passed

=== ALL TIMESTAMP RANGE TESTS PASSED! ? ===
DSA-6 Implementation Complete!
```

D. Sample Timestamp Range Query Output

Query: Interactions between 08:30 and 09:00 on 2025-11-23
Total interactions returned: 15

Sample results (first 3 of 15):
I016 U003 2025-11-23 08:31 PLAY C003 150
I017 U002 2025-11-23 08:33 LIKE C002 0
I018 U008 2025-11-23 08:35 PLAY C009 190

E. Longest Viewing Span Results (Dataset_1)

User ID	First Interaction	Last Interaction	Span MIn
U002	08:05	09:35	90 ✓ Winner
U001	08:01	09:25	84
U003	08:10	09:34	84
U004	08:15	09:31	76
U005	08:20	09:29	69
U006	08:25	09:26	61
U007	08:30	09:22	52
U008	08:35	09:18	43
U009	08:37	09:16	39
U010	08:39	09:09	30

Winner: U002 with 90-minute span